

Text Property Graph Construction and the Multiview GNN Model

1 Purpose

Each CVE record is converted into a Text Property Graph (TPG), and the multiview GNN consumes that graph to produce a prediction. This note walks through the construction pipeline, the model architecture, and the training settings. The goal is to be precise enough that an engineer can reimplement the model, but light enough on notation that a practitioner can read it in one sitting.

The model has two branches:

- the **graph branch**, which learns from the TPG structure (nodes, typed edges, contextual node features);
- the **tabular branch**, which learns from CVE-level metadata (CVSS, CWE, age, source counts, public-exploit availability).

Edge types are graph-branch features. The tabular branch never sees them.

2 TPG Construction Procedure

Each CVE graph is built from text. In the description-only baseline the text is the CVE description. In the summary-only variants the text is one of the LLM summary columns. In the combined (ALL) variant the text is the description plus all three summaries joined.

Construction proceeds in seven steps:

1. Pick the text source (description, one summary, or description + summaries combined).
2. Run the spaCy linguistic frontend. This produces document, paragraph, sentence, token, named-entity, predicate, argument, noun-phrase, verb-phrase and clause nodes, plus the base linguistic edges (DEP, NEXT_TOKEN, NEXT_SENT, NEXT_PARA, COREF, SRL_ARG, CONTAINS, BELONGS_TO, ENTITY_REL, RST_RELATION, DISCOURSE, SIMILARITY, AMR_EDGE).
3. Run the security frontend. This adds CVE, CWE, software, version, attack-vector, impact, severity and remediation entities on top of the linguistic graph. With `-include-security-edges`, the security-relations pass also emits ten typed `SEC_*` edges between those entities.
4. Compute a 768-dim contextual vector for every text-bearing node with SecBERT (jackaduma/SecBERT, frozen). SecBERT runs on WordPiece subtokens and the resulting subtoken vectors are averaged within the boundary of each node's text span (a token's own subtokens for token nodes; the constituent tokens for sentence, paragraph, entity, predicate, noun-phrase and clause nodes). Nodes without text get a zero vector. Every node ends up with the same 768-dim slot in its feature vector, regardless of type.

5. Convert the graph to PyTorch Geometric tensors: `x` (node features), `edge_index`, `edge_type`, and `edge_attr` (one-hot edge type).
6. Attach the target. In soft-label runs the target is the EPSS probability $\in [0, 1]$. In binary-mode runs it is either CISA KEV membership or the EPSS score thresholded at 0.1.

3 Graph Representation

After construction, a CVE graph has N nodes, M directed edges, and an edge-type vocabulary of size $|R|$. The vocabulary is $|R| = 13$ for the base schema, or $|R| = 23$ when security edges are turned on (the 13 base types plus the 10 `SEC_*` types).

Node features. Each node i has a feature vector

$$x_i = [\text{node-type one-hot (13 dims); SecBERT embedding (768 dims)}] \in \mathbb{R}^{781}.$$

If `-add-source-labels` is set, a 3-dim source one-hot (description, summary, mixed) is appended, giving 784 dims. The 13 node types are document, paragraph, sentence, token, entity, predicate, argument, concept, noun-phrase, verb-phrase, clause, mention, topic.

Edge tensors. The M edges are stored as two integer tensors:

- `edge_index`: a $2 \times M$ tensor of source and target node indices.
- `edge_type`: an M -vector of relation indices in the range 0 to $|R| - 1$. Indices 0 to 12 are the base types; indices 13 to 22 are the `SEC_*` types when security edges are enabled.

4 Multiview Edge Partition

A single-view GNN treats every edge identically. The multiview decomposition splits the edge vocabulary into disjoint groups called views, each capturing a different kind of relation. The base configuration uses four views; a fifth view is added when the security edges are enabled.

View	Edge types in the view
Syntactic	DEP, CONTAINS, BELONGS_TO
Sequential	NEXT_TOKEN, NEXT_SENT, NEXT_PARA
Semantic	COREF, SRL_ARG, AMR_EDGE
Discourse	RST_RELATION, DISCOURSE, ENTITY_REL, SIMILARITY
Security	all <code>SEC_*</code> types (only present when $ R = 23$)

Table 1: The four base views plus the optional security view, with their edge types.

For each view v , we keep only that view’s edges. Call the resulting edge subset E_v . When security edges are off, E_{security} is empty and that view’s encoder acts as the identity (no information flows through it).

5 Per-View Encoder

Before any view-specific processing, the model lifts every node feature into a hidden space of dimension $h = 128$ with a small input block: a linear projection W_{in} , followed by LayerNorm, GELU and Dropout. The projected node features are written $H^{(0)}$.

Each view has its own encoder, which is a Gated Graph Neural Network (GGNN; Li et al., 2015). The encoder runs K message-passing steps restricted to that view’s edges:

$$H_v = \text{ViewEncoder}_v(H^{(0)}, E_v) \in \mathbb{R}^{N \times h}.$$

A single GGNN step is a standard message-passing update: each node sums linear messages from its in-neighbours, then a GRU cell combines that sum with the node’s previous hidden state. The same update is unrolled K times so a node’s representation can pick up information from neighbours up to K hops away. We add a residual connection plus a LayerNorm and Dropout around the encoder, and use $K = 3$ in all reported runs.

6 Node-Conditioned View Fusion

Each node now has V candidate embeddings, one per active view ($V = 4$ in the base configuration, $V = 5$ when the security view is on). To combine them, we use a small attention mechanism that picks how much weight each view gets, separately for each node.

For node i , the model computes a query vector q_i from the input projection and a key vector k_i^v from each view’s embedding. Both vectors live in $\mathbb{R}^h$. The model scores each view by the scaled dot product $q_i \cdot k_i^v / \sqrt{h}$, normalises the V scores across views with a softmax to get attention weights α_i^v , and forms the fused embedding as the weighted sum:

$$h_i = \sum_v \alpha_i^v H_v[i, :].$$

Because the attention weights are computed per node, different nodes can rely on different views. A token inside a security entity can lean on the security view; a discourse connective can lean on the discourse view; a verb in a predicate-argument structure can lean on the semantic view.

7 Graph Pooling and Prediction Head

The N fused node embeddings are pooled into a single graph vector g by concatenating their element-wise mean and their element-wise maximum across nodes. With $h = 128$, g has $2h = 256$ dimensions. A small MLP head then turns g into a single logit $\hat{\ell}$, and the sigmoid $\sigma(\hat{\ell})$ is the prediction. In the hybrid configuration the head’s input is the concatenation of g with a 64-dim tabular embedding produced by a separate tabular MLP.

8 Training Objective and Optimisation

The loss is binary cross-entropy with logits between the predicted probability $\sigma(\hat{\ell})$ and the target y . Positive examples are up-weighted by a factor equal to the negative-to-positive ratio in the training set, which keeps the rare high-EPSS class from being drowned out. The same loss handles both hard $\{0, 1\}$ labels (KEV membership) and soft $[0, 1]$ labels (raw EPSS scores), since BCE is defined for any target in $[0, 1]$.

The optimiser is AdamW. Gradient norms are clipped at 1.0 after each backward pass. The learning rate is reduced on plateau (mode max, factor 0.5, patience 5, minimum learning rate 10^{-6}), and training stops early after 15 epochs without an improvement in validation PR-AUC. The threshold 0.1 on EPSS is used only for stratified splitting and for binarising the labels at metric time (precision, recall and F1 at decision threshold 0.5). It is never the training target.

9 Summary of Hyperparameters

The values used in the 20-run text-source ablation and the NVD/KEV runs are collected here so they live next to the architecture description.

Component	Setting	Value
Backbone	-backbone	multiview
Hidden dimension	h	128
Tabular MLP width	h_t	64
GGNN message-passing steps	K (-layers)	3
Dropout	-dropout	0.3
Epochs	-epochs	100
Batch size	-batch-size	32
Learning rate	-lr	1×10^{-3}
Weight decay	-weight-decay	1×10^{-4}
Gradient-norm clip		1.0
Scheduler	reduce-on-plateau	mode max, factor 0.5, patience 5, min lr 10^{-6}
Early-stopping patience	-patience	15 epochs without val PR-AUC gain
Random seed	-seed	42
SecBERT embedding dim	-embedding-dim	768
Top-K CWEs (NVD/KEV)	-top-k-cwes	25

Table 2: Hyperparameters used in the reported experiments.

10 What the Architecture Actually Encodes

Edge labels are not just counted. Their placement in the graph is part of the model input. A **SEC_AFFECTS** edge tells the GNN which CVE node connects to which software node. A tabular one-hot feature could only record that such a relation exists somewhere, losing the structural information.

For this reason **edge_index** together with **edge_type** participate in typed message passing through the per-view GGNN encoders, while tabular features carry only document-level metadata.

End-to-end, the graph branch turns X , **edge_index** and **edge_type** into a 256-dim graph vector g via input projection, V per-view GGNN encoders, node-conditioned view fusion, and mean-plus-max pooling. The tabular branch turns the tabular feature vector into a 64-dim embedding via a small MLP. In the hybrid configuration the prediction is the sigmoid of a fusion MLP applied to the concatenation of the graph vector and the tabular embedding; in the graph-only configuration the prediction comes from g alone.